CDAC PGCP-HPCSA

Cloud Services and Security

Practical Examination – Set B | Complete Solutions

Question	Practical topic	Marks
Q1	Jenkins Freestyle + Java + Docker	10
Q2	Jenkins Pipeline + Python + Docker	10
Q3	Docker image tags and container execution	10
Q4	Git repository, branches, commits and merge	10
Total		40

How to use this guide: Replace placeholders such as **YOUR_GITHUB_USERNAME** and **PRN** with your own values. Execute commands in order and capture screenshots at the checkpoints listed in each question.

Assumptions: Ubuntu/Linux terminal, Git, Docker Engine, Jenkins, and a GitHub account are available. Jenkins must be able to run Git and Docker commands on its build agent.

Exam overview

Q	What you must demonstrate	Main evidence
1	Freestyle job checks out Java project, builds image, runs container	GitHub repo + Jenkins Console Output
2	Pipeline checks out Python project, builds image, runs container	Jenkinsfile + Pipeline stage logs
3	Build one image with three tags, remove one tag, run another tag	docker images + run output
4	Create main/dev branches, make commits, merge and show history	git log --oneline --graph + file content

Before starting

```
git --version
docker --version
docker info
java -version
python3 --version
```

If Docker daemon is stopped on Ubuntu:

```
sudo systemctl start docker
sudo systemctl enable docker
```

> If Jenkins cannot access Docker, add its service account to the docker group and restart Jenkins. This grants powerful host-level access, so do this only on your own lab machine.

```
sudo usermod -aG docker jenkins
sudo systemctl restart jenkins
```

If permission still fails, log out/restart the Jenkins agent or service session so the new group membership is applied. Do not use chmod 777 on the Docker socket.

Q1. Jenkins Freestyle Project – Java multiplication (10 Marks)

Task: Create a GitHub repository containing a Java program that multiplies two numbers and a Dockerfile. Configure a Jenkins Freestyle project to fetch the code from GitHub, build a Docker image, run a Docker container, and verify the output.

1. Create the GitHub repository

• Create a public GitHub repository named **java-multiply**.

• Add two files at the repository root: **Main.java** and **Dockerfile**.

2. Main.java

```
public class Main {
    public static void main(String[] args) {
        int num1 = 10;
        int num2 = 20;
        int result = num1 * num2;

        System.out.println("Number 1: " + num1);
        System.out.println("Number 2: " + num2);
        System.out.println("Multiplication: " + result);
    }
}
```

3. Dockerfile

```
FROM eclipse-temurin:17-jdk
WORKDIR /app
COPY Main.java .
RUN javac Main.java
CMD ["java", "Main"]
```

The image contains a Java 17 JDK, copies the source file into /app, compiles it while building the image, and runs the compiled class when the container starts.

4. Push the project to GitHub

```
git init
git add .
git commit -m "Initial Java Docker project"
git branch -M main
git remote add origin https://github.com/YOUR_GITHUB_USERNAME/java-multiply.git
git push -u origin main
```

If the remote already exists, use **git remote set-url origin URL** instead of adding it again.

5. Create and configure the Jenkins Freestyle project

• Open Jenkins (commonly http://localhost:8080) and select **New Item**.

• Enter **Java-Multiply-Freestyle**, select **Freestyle project**, and click OK.

• Under **Source Code Management**, select **Git**.

• Repository URL: **https://github.com/YOUR_GITHUB_USERNAME/java-multiply.git**.

• Branch Specifier: ***/main**. For a public repository, credentials can usually be left empty.

• Under **Build Steps**, select **Execute shell** and paste the commands below.

```
set -e
echo "Starting Java Docker build"
docker build -t java-multiply:latest .
echo "Running Java container"
docker run --rm java-multiply:latest
echo "Build completed successfully"
```

Click Save, then click **Build Now**. Open the build number and select **Console Output**.

6. Expected output

```
Starting Java Docker build
...
Running Java container
Number 1: 10
Number 2: 20
Multiplication: 200
Build completed successfully
Finished: SUCCESS
```

7. Verification and screenshots

```
docker images
docker run --rm java-multiply:latest
```

• Screenshot 1: GitHub repository showing Main.java and Dockerfile.

• Screenshot 2: Jenkins Freestyle project configuration showing Git repository and branch.

• Screenshot 3: Execute shell build step.

• Screenshot 4: Jenkins Console Output with Multiplication: 200 and Finished: SUCCESS.

• Screenshot 5 (optional): docker images output showing java-multiply:latest.

Important: Jenkins must run these commands on an agent that has Docker installed and permission to use the Docker daemon. If Docker is installed only on another machine, configure a suitable agent or Docker connection.

Q2. Jenkins Pipeline – Python square (10 Marks)

Task: Create a GitHub repository containing a Python program that calculates the square of a number and a Dockerfile. Create a Jenkins Pipeline using a Jenkinsfile to fetch the code from GitHub, build a Docker image, run a Docker container, and verify the output.

1. Create GitHub repository and files

Create a repository named **python-square** with these files at its root:

```
python-square/
■■■ square.py
■■■ Dockerfile
■■■ Jenkinsfile
```

2. Python program – square.py

```
number = 7
result = number * number

print("Number:", number)
print("Square:", result)
```

3. Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY square.py .
CMD ["python", "square.py"]
```

4. Jenkinsfile (Declarative Pipeline)

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t python-square:latest .'
            }
        }

        stage('Run Docker Container') {
            steps {
                sh 'docker run --rm python-square:latest'
            }
        }
    }

    post {
        success {
            echo 'Pipeline completed successfully!'
        }
        failure {
            echo 'Pipeline failed. Check the stage logs.'
        }
    }
}
```

Note: checkout scm checks out the repository configured for the Pipeline job. It is suitable when the Pipeline is created from the same Git repository (Pipeline script from SCM).

5. Push files to GitHub

```
git init
git add .
git commit -m "Add Python square Docker pipeline"
git branch -M main
git remote add origin https://github.com/YOUR_GITHUB_USERNAME/python-square.git
```

```
git push -u origin main
```

6. Create Jenkins Pipeline job

• In Jenkins, click **New Item**.

• Enter **Python-Square-Pipeline** and select **Pipeline**.

• Under Pipeline, set Definition to **Pipeline script from SCM**.

• SCM: **Git**.

• Repository URL: **https://github.com/YOUR_GITHUB_USERNAME/python-square.git**.

• Branch: ***/main**.

• Script Path: **Jenkinsfile**.

• Click Save, then **Build Now**.

7. Expected Console Output

```
[Pipeline] stage
[Pipeline] { (Checkout)
...
[Pipeline] { (Build Docker Image)
...
Successfully built / tagged python-square:latest
[Pipeline] { (Run Docker Container)
Number: 7
Square: 49
Pipeline completed successfully!
Finished: SUCCESS
```

Exact Docker build log wording varies by Docker version. The key verification is that the container prints Number: 7 and Square: 49, and the Pipeline finishes successfully.

8. Screenshots to capture

• GitHub repository showing square.py, Dockerfile and Jenkinsfile.

• Jenkins Pipeline configuration showing Pipeline script from SCM and repository URL.

• Pipeline Stage View showing Checkout, Build Docker Image and Run Docker Container.

• Console Output showing Number: 7, Square: 49 and Finished: SUCCESS.

Q3. Docker image tagging and container execution (10 Marks)

Task from the paper: Write a Python program that prints “Welcome to HPCSA - ”; create a Dockerfile using python:3.11-slim and copy the Python script into the image; build the image with three tags (latest, v1 and stable); list images; remove the stable tag without deleting the image; run the image with tag v1 and show the output.

1. Create Python program

Create a working directory and move into it. Replace PRN with your actual PRN.

```
mkdir q3-docker
cd q3-docker
nano welcome.py
```

```
print("Welcome to HPCSA - YOUR_PRN")
```

2. Create Dockerfile

```
nano Dockerfile
```

```
FROM python:3.11-slim
WORKDIR /app
COPY welcome.py .
CMD ["python", "welcome.py"]
```

3. Build image with three tags

One docker build command can create multiple tags for the same image:

```
docker build -t hpcsa-welcome:latest \
             -t hpcsa-welcome:v1 \
             -t hpcsa-welcome:stable .
```

4. List Docker images

```
docker images
```

Expected: the repository **hpcsa-welcome** should be listed with tags **latest**, **v1** and **stable**. The IMAGE ID should be the same for all three tags.

5. Remove only the stable tag

```
docker rmi hpcsa-welcome:stable
```

This removes the local tag reference. Because the same image is still referenced by latest and v1, the image data remains available.

6. Verify remaining tags

```
docker images hpcsa-welcome
```

Expected tags: **latest** and **v1**. The stable tag should no longer appear.

7. Run the image with v1 tag

```
docker run --rm hpcsa-welcome:v1
```

Expected output:

```
Welcome to HPCSA - YOUR_PRN
```

Replace YOUR_PRN with your actual PRN in the Python script before building the image.

Screenshots to capture

• Python script and Dockerfile.

• Successful docker build command with all three tags.

• docker images output showing latest, v1 and stable.

- docker rmi hpcsa-welcome:stable command.
- docker images output showing latest and v1 only.
- docker run output showing Welcome to HPCSA - YOUR_PRN.

Q4. Git repository, branches, commits and merge (10 Marks)

Task from the paper: Initialize a local Git repository; create a blank file named .txt and commit it to main; create branch dev_ and add “Line from dev” to the file and commit; switch to main, edit the same line to “Line from main branch” and commit; merge dev_ into main, then show git log --oneline --graph and the content of .txt.

Important: The paper asks both branches to change the same file line. This can produce a merge conflict. The steps below show how to resolve it and complete the merge.

1. Initialize local repository and create blank file

Use your PRN in the filename. Example below uses 210940130001; replace it with your own.

```
mkdir q4-git
cd q4-git

git init
git branch -M main

touch 210940130001.txt
git add 210940130001.txt
git commit -m "Create blank PRN file"
```

If Git asks for your identity, configure it:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

2. Create dev branch and add its line

```
git switch -c dev_210940130001
echo "Line from dev" > 210940130001.txt
git add 210940130001.txt
git commit -m "Add line from dev branch"
```

3. Switch to main and edit the same line

```
git switch main
echo "Line from main branch" > 210940130001.txt
git add 210940130001.txt
git commit -m "Update line from main branch"
```

4. Merge dev branch into main

```
git merge dev_210940130001
```

Because both branches changed the same line differently, Git may report a content conflict. If it does, open the file:

```
cat 210940130001.txt
```

You may see conflict markers similar to:

```
<<<<<<< HEAD
Line from main branch
=======
Line from dev
>>>>>>> dev_210940130001
```

Resolve the conflict by editing the file and keeping the content required for the final submission. For example, keep both lines:

```
Line from main branch
Line from dev
```

Then stage and complete the merge:

```
git add 210940130001.txt
git commit -m "Merge dev branch into main"
```

If Git performs a fast-forward or automatically completes the merge without conflict, do not create an extra merge commit. Continue to the verification commands.

5. Show Git history and file content

```
git log --oneline --graph --all
echo "----- File Content -----"
cat 210940130001.txt
git status
```

Expected: the log shows the initial commit, the dev commit, the main commit and (if a conflict was resolved) the merge commit. The file displays the final resolved lines. The working tree should be clean.

Screenshots to capture

• git init and initial blank-file commit on main.

• git switch -c dev_PRN and dev commit.

• Switch to main and main commit.

• Merge command and conflict resolution, if a conflict occurs.

• git log --oneline --graph --all output.

• cat PRN.txt showing final content.

Final submission checklist

Question	Evidence to include
Q1	GitHub Java repository, Freestyle configuration, successful Console Output
Q2	GitHub Python repository with Jenkinsfile, Pipeline stages and successful output
Q3	Three tags, stable tag removed, v1 container output
Q4	Initial/dev/main commits, merge result, graph log and final file content

Useful Docker commands

```
docker images
docker ps
docker ps -a
docker build -t name:tag .
docker run --rm name:tag
docker rmi name:tag
docker image inspect name:tag
```

Useful Git commands

```
git status
git branch
git branch -a
git switch main
git switch -c branch_name
git add .
git commit -m "message"
git merge branch_name
git log --oneline --graph --all
git diff
git show --stat
```

Exam submission reminder: Number screenshots according to the question and step (for example Q1-1, Q1-2, Q2-1). Keep all Q1–Q4 folders inside one parent folder if your exam instructions require one ZIP. Close files and terminals only after confirming the screenshots and outputs have been saved.

This is a practice solution guide. Adapt repository names, PRN, hostnames, credentials and Jenkins setup to the lab environment provided during the examination.